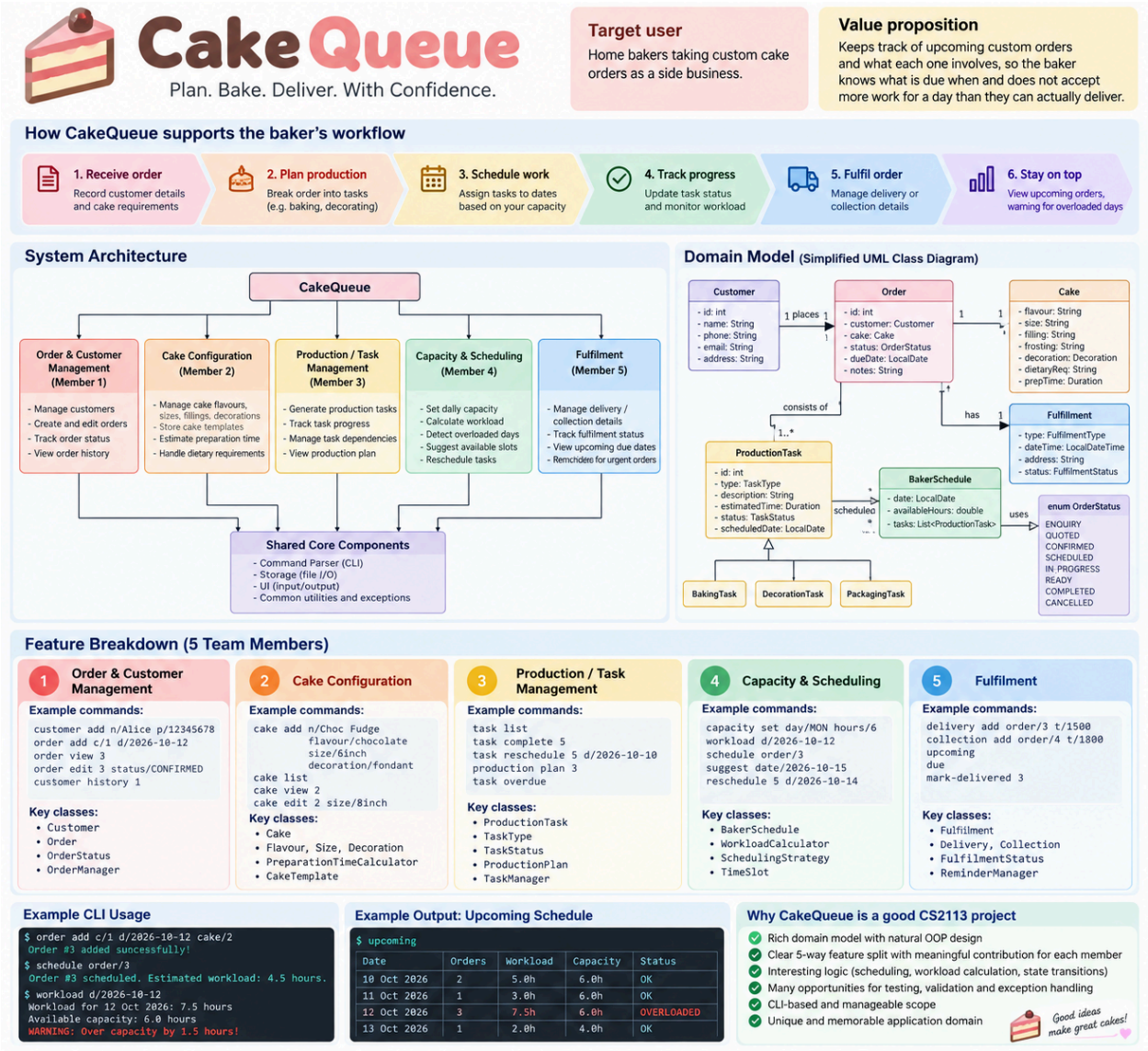


I actually think **CakeQueue is a very good CS2113 project idea**, especially for a 5-person team. It is more distinctive than a generic inventory/task manager, and the domain naturally gives you objects, rules, state transitions, and scheduling logic.

The key question is whether you can make it **deep enough for 5 people without turning it into an unnecessarily large bakery-management system**. I think you can.

Why CakeQueue works well

Your current idea already has a strong target user and value proposition:

> **Target user:** home bakers taking custom cake orders as a side business
> **Value proposition:** keeps track of upcoming custom orders and what each one involves, so the baker knows what is due when and does not accept more work for a day than they can actually deliver.

That is much better than saying "an app for managing cake orders."

The important part is **"what each one involves"** and **"does not accept more work for a day than they can actually deliver."**

Those two phrases give you substantial domain logic.

For example, an order could contain:

Order

├── Customer

├── Cake

│ ├── Flavour

│ ├── Size

│ ├── Filling

│ └── Decorations

│

├── Tasks

│ ├── Baking

│ ├── Filling

│ ├── Decoration

│ └── Packaging

│

└── Delivery / Collection

Now the system isn't just:

add order

delete order

list order

It becomes a **production planning system for a one-person bakery**.

The biggest opportunity: workload planning

I think this should become the **central selling point** of CakeQueue.

Suppose the baker has:

Monday

- Bake chocolate cake: 2h

- Prepare buttercream: 1h

- Decorate wedding cake: 4h

- Package cupcakes: 30m

Available capacity: 6h

Total workload: 7.5h

CakeQueue should tell them:

> **WARNING: Monday is over capacity by 1h 30m.**

This immediately gives you interesting algorithms and business rules.

For example:

Order A

Due: 14 Sep

Baking: 2h

Decoration: 3h

Packaging: 30m

Order B

Due: 14 Sep

Baking: 1h

Decoration: 2h

Packaging: 30m

The system could calculate:

14 Sep workload = 9 hours

Baker capacity = 6 hours

Overloaded = 3 hours

That is much more interesting from an OOP perspective than simply storing orders.

It can naturally support 5 contributors

This is where I think CakeQueue becomes particularly attractive for your team.

I'd divide it like this:

Member 1 — Order & Customer Management

Responsible for:

order add

order edit

order cancel

order view

order list

customer add

customer history

Classes:

Order

Customer

CakeOrder

OrderStatus

OrderRepository

Possible states:

PENDING

CONFIRMED

IN_PROGRESS

READY

COMPLETED

CANCELLED

Member 2 — Cake Configuration

This person handles the actual cake requirements.

For example:

cake add

cake edit

cake view

A cake might consist of:

Cake

├── Flavour

├── Size

├── Filling

├── Frosting

├── Decoration

└── DietaryRequirement

This can give you some natural abstraction.

For example:

Decoration

├── FondantDecoration

├── ButtercreamDecoration

├── ChocolateDecoration

└── FreshFruitDecoration

Each could have different estimated preparation time or storage requirements.

This is a much more defensible use of inheritance/polymorphism than inventing arbitrary `CakeA`, `CakeB`, `CakeC` subclasses.

Member 3 — Production / Task Management

This could be one of the most substantial components.

Each order gets a production plan:

Order #32

1. Bake sponge 2h

2. Prepare filling 1h

3. Assemble 30m

4. Decorate 3h

5. Package 15m

Commands:

task list

task complete 12

task postpone 12

task schedule 12 date/2026-09-15

production view date/2026-09-15

Classes:

ProductionTask

├── BakingTask

├── PreparationTask

├── DecorationTask

└── PackagingTask

ProductionPlan

TaskStatus

This gives you excellent object interaction.

Member 4 — Capacity & Scheduling

This is probably your **most technically interesting subsystem**.

The baker defines their capacity:

capacity set day/MON hours/6

capacity set day/TUE hours/4

Then CakeQueue calculates:

daily workload

remaining capacity

overloaded dates

available slots

Potential classes:

BakerSchedule

WorkDay

TimeSlot

Capacity

WorkloadCalculator

SchedulingStrategy

You could even have:

SchedulingStrategy

├── DeadlineFirstStrategy

└── EarliestAvailableStrategy

Again, only implement this kind of abstraction where it genuinely helps.

Member 5 — Delivery / Collection & Notifications

The final member can handle order fulfilment logistics.

For example:

delivery add order/32 time/1500

collection add order/33 time/1800

upcoming

due

overdue

Classes:

Fulfilment

├── Delivery

└── Collection

DeliveryAddress

TimeSlot

Reminder

They can integrate heavily with the scheduling subsystem:

Order due at 3pm

↓

Tasks must be completed beforehand

↓

Packaging deadline

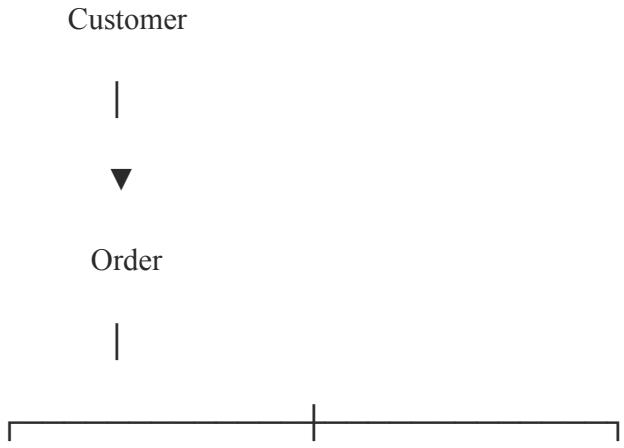
↓

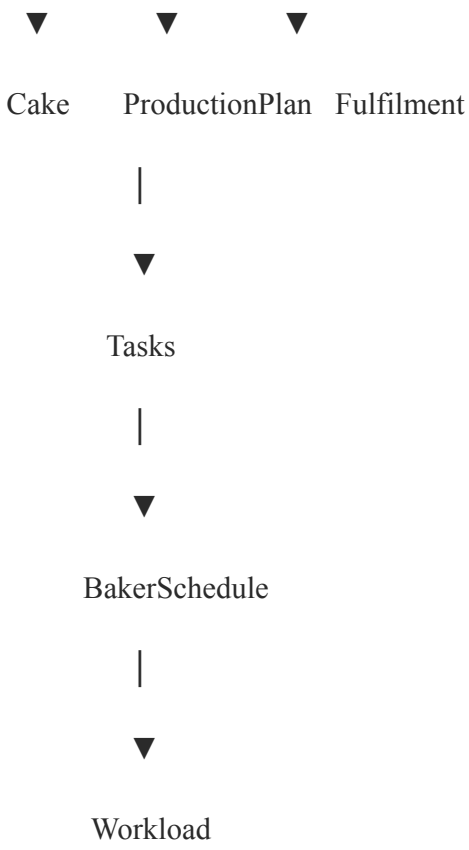
Delivery / collection slot

That means all five components remain connected.

This gives you a very nice architecture

Conceptually:





The really nice part is that there is a **central domain concept: Order**.

Almost everyone interacts with it, but nobody needs to own the entire application.

There is also a great opportunity for state transitions

Custom cake orders naturally move through stages.

For example:

ENQUIRY

↓

QUOTED

↓

CONFIRMED

↓

SCHEDULED

↓

IN PROGRESS

↓

READY

↓

DELIVERED

But you can impose rules such as:

Cannot start production

→ unless order is CONFIRMED

Cannot mark READY

→ unless required tasks are complete

Cannot mark DELIVERED

→ unless order is READY

These rules create excellent opportunities for:

- encapsulation
- validation
- exception handling

- unit testing
- sequence diagrams
- state modelling

That is exactly the kind of complexity I'd prefer over simply adding lots of commands.

Your value proposition can be made even stronger

I'd slightly refine the wording.

Your current:

> "Keeps track of upcoming custom orders and what each one involves, so the baker knows what is due when and does not accept more work for a day than they can actually deliver."

is already good, but I would make the **capacity-management problem** more explicit.

Something along these lines is stronger:

> **CakeQueue helps home bakers manage custom cake orders by breaking each order into production tasks, scheduling work against their daily capacity, and highlighting overloaded days before they accept too much work.**

Now you have a very clear problem:

Home bakers don't just need to know what orders exist; they need to know whether they can realistically fulfil them.

That's a genuine workflow.

One thing I'd be careful about

Don't let CakeQueue turn into:

Cake shop POS

+ inventory system

+ accounting

+ customer CRM

+ delivery tracking

+ recipe manager

+ online ordering

+ payment system

That would be far too much.

I would explicitly **not** make inventory, payments, online customer ordering, etc. core features unless they become necessary to your central workflow.

Your core loop should remain:

CUSTOMER ORDER

↓

CAKE REQUIREMENTS

↓

PRODUCTION TASKS

↓

SCHEDULE

↓

CAPACITY CHECK

↓

FULFILMENT

That is already enough for a very solid 5-person project.

OOP potential: 9/10

I'd rate CakeQueue approximately:

Criterion	Rating	Why
OOP opportunities	⭐⭐⭐⭐⭐	Orders, cakes, tasks, schedules, fulfilment, states
5-person workload	⭐⭐⭐⭐⭐	Very natural 5-way feature split
Testability	⭐⭐⭐⭐⭐	Lots of calculations and validation rules
CLI suitability	⭐⭐⭐⭐⭐	Scheduling/query commands work well in CLI
Domain uniqueness	⭐⭐⭐⭐⭐	More memorable than generic managers
Scope control	⭐⭐⭐⭐	Good, provided you resist adding an entire bakery ERP
Risk	⭐⭐⭐⭐	Scheduling logic can get complex, but manageable

So yes: **I would absolutely keep CakeQueue on your shortlist.**

In fact, compared with the ideas I suggested earlier, I think **CakeQueue is potentially a better CS2113 project for a 5-person team**, because the domain gives you a clear central workflow while still allowing five people to own meaningful components.

The one thing I'd establish before committing is a **feature boundary for v1.0** so that every member has roughly comparable implementation + testing + documentation work. I can also create an image explaining the CakeQueue architecture and how the 5 members' components interact.

CakeQueue User Stories

Order & Customer Management

- **[Must]** As a baker, I can add a new order with customer name, contact, and due date, so that I have a record of what's been requested.
- **[Must]** As a baker, I can view the full details of a specific order, so that I can recall exactly what was agreed.
- **[Must]** As a baker, I can list all orders, so that I can see everything on my plate at a glance.
- **[Must]** As a baker, I can edit an order's details before it's confirmed, so that I can accommodate changes the customer requests.
- **[Must]** As a baker, I can cancel an order, so that it no longer counts toward my workload.
- **[Should]** As a baker, I can view a customer's order history, so that I can recognize repeat customers and their preferences.
- **[Could]** As a baker, I can search orders by customer name or due date, so that I can quickly find a specific order among many.
- **[Could]** As a baker, I can tag an order as a repeat/favourite customer, so that I can prioritize their requests.

Cake Configuration

- **[Must]** As a baker, I can specify a cake's flavour, size, and filling when creating an order, so that the requirements are captured accurately.
- **[Must]** As a baker, I can add decoration details to a cake, so that I know what decorating work is involved.
- **[Must]** As a baker, I can edit a cake's configuration before production starts, so that last-minute customer changes are reflected.
- **[Should]** As a baker, I can record dietary requirements (e.g. gluten-free, nut-free, vegan) for a cake, so that I don't accidentally use the wrong ingredients.
- **[Should]** As a baker, I can see the estimated prep time for a given decoration type, so that I can gauge how long the order will take before accepting it.
- **[Could]** As a baker, I can save a cake configuration as a template, so that I can quickly reuse common combinations for future orders.

Production / Task Management

- **[Must]** As a baker, I can view the list of production tasks generated for an order, so that I know exactly what steps remain.
- **[Must]** As a baker, I can mark a task as complete, so that I can track my progress through an order.
- **[Must]** As a baker, I can see which tasks are still pending for a given day, so that I know what to work on next.

- **[Should]** As a baker, I can postpone a task to a later date, so that I can adjust my plan if something runs behind.
- **[Should]** As a baker, I can view all tasks scheduled across all orders for a specific day, so that I get a full picture of my day's work.
- **[Could]** As a baker, I can reorder tasks within an order, so that I can adapt my workflow to how I actually like to bake.

Capacity & Scheduling

- **[Must]** As a baker, I can set my available working hours for each day of the week, so that the system knows my real capacity.
- **[Must]** As a baker, I can see the total workload scheduled for a given day, so that I know how much work I've committed to.
- **[Must]** As a baker, I can be warned when a day's workload exceeds my set capacity, so that I don't overcommit.
- **[Should]** As a baker, I can see which upcoming days still have free capacity, so that I know when I can safely accept a new order.
- **[Should]** As a baker, I can view a week-at-a-glance summary of workload vs. capacity, so that I can plan further ahead.
- **[Could]** As a baker, I can choose between a "deadline-first" or "earliest-available" scheduling strategy, so that tasks get automatically slotted in a way that fits how I prefer to work.

Delivery / Collection & Fulfilment

- **[Must]** As a baker, I can set a delivery or collection time for an order, so that I know when the cake needs to be ready.
- **[Must]** As a baker, I can view all upcoming deliveries/collections, so that I don't forget any handoffs.
- **[Must]** As a baker, I can mark an order as delivered/collected, so that my records reflect completed work.
- **[Should]** As a baker, I can see overdue orders that were not delivered/collected on time, so that I can follow up with the customer.
- **[Should]** As a baker, I can be prevented from marking an order "delivered" if its tasks aren't all complete, so that I don't hand over an unfinished cake by mistake.
- **[Could]** As a baker, I can set a reminder ahead of a delivery time, so that I don't lose track of a slot on a busy day.

Entire System

- **[Must]** As a baker, I can save my data automatically after every change, so that I don't lose my orders if the app closes unexpectedly.
- **[Must]** As a baker, I can load my saved data when I restart the app, so that I don't have to re-enter everything.

- **[Should]** As a new user, I can access a help command listing all available commands, so that I can learn how to use the app without external documentation.
- **[Could]** As a baker, I can undo my last action, so that I can recover quickly from an accidental command

A. Managing orders

1. **As a user, I can add an order, so that I can keep track of my commitments.**
2. **As a user, I can remove an order, so that cancelled orders no longer affect my planning.**
3. **As a user, I can update an order, so that I can account for changes requested by a customer.**
4. **As a user, I can view an order, so that I can recall its details.**
5. **As a user, I can view all upcoming orders, so that I know what I have committed to.**
6. **As a user, I can find a specific order, so that I can access it quickly when needed.**
7. **As a user, I can record who an order is for, so that I know which customer it belongs to.**
8. **As a user, I can record when an order is due, so that I know when it needs to be completed.**

B. Recording cake requirements

9. **As a user, I can record the size of a cake, so that I remember what the customer requested.**
10. **As a user, I can record the flavour of a cake, so that I remember what I need to prepare.**
11. **As a user, I can record decoration requirements, so that I remember how the cake should look.**
12. **As a user, I can record dietary requirements, so that I do not overlook important customer needs.**
13. **As a user, I can record special instructions, so that unusual customer requests are not forgotten.**
14. **As a user, I can see all requirements associated with an order, so that I can prepare it correctly.**

C. Planning work

15. **As a user, I can record the work needed for an order, so that I know what needs to be completed.**
16. **As a user, I can estimate how much time a piece of work will take, so that I can plan my workload.**
17. **As a user, I can see unfinished work, so that I know what remains to be done.**
18. **As a user, I can mark work as completed, so that I can keep track of my progress.**
19. **As a user, I can see what work needs my attention first, so that I know what to work on next.**
20. **As a user, I can see work that is overdue, so that I can address delays quickly.**

D. Managing workload

21. **As a user, I can record how much time I am available to work, so that my planning reflects my actual availability.**
22. **As a user, I can see how much work I have planned for a day, so that I know how busy I will be.**
23. **As a user, I can see whether I have too much work planned for a day, so that I can avoid overcommitting myself.**
24. **As a user, I can see how much more work I can take on, so that I can decide whether to accept another order.**
25. **As a user, I can see my busiest upcoming days, so that I can prepare for periods of high workload.**
26. **As a user, I can rearrange my planned work, so that I can adapt when my availability changes.**
27. **As a user, I can assess the impact of a potential order on my workload, so that I can decide whether to accept it.**

E. Tracking orders

28. **As a user, I can see the current progress of an order, so that I know how close it is to completion.**
29. **As a user, I can mark an order as completed, so that I know I no longer need to work on it.**
30. **As a user, I can see orders that are due soon, so that I can prioritise them.**
31. **As a user, I can see orders that are overdue, so that I can address them urgently.**
32. **As a user, I can distinguish completed orders from active orders, so that I can focus on current commitments.**

F. Fulfilment

33. **As a user, I can record whether an order will be collected or delivered, so that I know how it will reach the customer.**
34. **As a user, I can record when an order will be collected or delivered, so that I know when it must be ready.**
35. **As a user, I can see orders that need to be collected or delivered soon, so that I can prepare them in time.**
36. **As a user, I can mark an order as fulfilled, so that I know the commitment is finished.**

Order Management

1. **Add order** — create a new order record (customer name, due date, at minimum)
2. **View order** — display full details of a single order
3. **List orders** — display all upcoming/active orders
4. **Update order** — edit an existing order's fields
5. **Cancel order** — remove/deactivate an order so it no longer counts toward workload
6. **Search orders** — find an order by keyword (likely customer name or order ID)

Customer Tracking

7. **Record customer name per order** — capture who the order is for
8. **View customer order history** — list past orders tied to a given customer name
9. **Identify repeat customers** — flag/detect when a customer name matches a previous order

Cake Details

10. **Record due date** — capture and store the date an order must be completed by
11. **Record cake size** — capture size as an order attribute
12. **Record cake flavour and filling** — capture flavour/filling as order attributes

Progress Tracking

13. **Mark work as completed** — flag/update an order (or its work) as done

Ideas for TP

1. DiveLog

- Target user: recreational scuba divers who dive at least 20 times a year
- Value proposition: Records dive details quickly after each dive and keeps a divers logged history in one place, so they can review past dives and track experience without carrying a paper logbook or wading through a bloated app.

2. CoachDesk

- Target user: strength coaches managing around 15 remote clients
- Value proposition: Captures each client's training session as it is reported and surfaces how their work is progressing over time, so the coach can prepare the next session without digging through scattered spreadsheets and chat messages.

3. QSOBook

- Target user: amateur radio operators who log contacts during live operating sessions
- Value proposition: Records radio contacts fast enough to keep up while still operating, and keeps the operator's contact history searchable afterwards, replacing paper logs that are slow to write and tedious to look things up in later.

4. CakeQueue

- Target user: home bakers taking custom cake orders as a side business
- Value proposition: Keeps track of upcoming custom orders and what each one involves, so the baker knows what is due when and does not accept more work for a day than they can actually deliver.

5. SendTracker

- Target user: recreational boulderers who climb regularly at one indoor gym
- Value proposition: Records attempts and sends on gym problems as the climber works through them, giving a clear picture of what they are currently projecting and how their climbing is progressing across grades.

6. ChainWatch

- Target user: daily bicycle commuters who maintain their own bikes
- Value proposition: Tracks distance ridden against the wear life of each bike component, so the commuter knows what needs replacing before something fails mid-commute, instead of guessing or discovering the problem too late.

7. BookCircle

- Target user: facilitators who organise a monthly in-person book club
- Value proposition: Keeps the group's reading schedule and members' progress in one place, so the facilitator knows what is being read, who chose it, who has finished, and when the next discussion falls.

8. WordBill

- Target user: freelance translators who charge per word
- Value proposition: Keeps translation jobs and their billing status in order, so the translator always knows what is owed, what is overdue, and what is due next, without maintaining a separate invoice spreadsheet.

CakeQueue

Target user: Home bakers taking custom cake orders as a side business

Value proposition: Keeps track of upcoming custom orders and what each one involves, so the baker knows what is due when and does not accept more work for a day than they can actually deliver.

User Stories:

Getting started

1. As a potential user exploring the app, I can see it pre-loaded with sample orders, so that I can see what a full queue looks like before typing anything.
2. As a user ready to start for real, I can purge all sample data, so that practice orders don't get mixed up with real ones.
3. As a new user, I can see a list of available commands, so that I can learn the app without hunting for a manual.
4. As a new user, I can record how many baking hours I have on each day of the week, so that the app knows what a normal day looks like for me.
5. As a new user, I can enter orders I already accepted elsewhere, so that the app reflects my real queue from day one.

Recording an order

6. As a baker, I can add an order with a customer name, cake description and due date, so that I stop relying on memory and screenshots.

7. As a baker, I can record the collection date *and time*, so that I know when the cake must actually be finished, not just which day.
8. As a baker, I can record the size and number of tiers, so that I can judge how much work the cake is.
9. As a baker, I can record flavour, filling and design notes, so that I make the cake the customer actually asked for.
10. As a baker, I can record the inscription text, so that I don't misspell a name on the day.
11. As a baker, I can record allergies and dietary restrictions, so that I don't put a customer at risk.
12. As a baker, I can record the quoted price and deposit received, so that I know what is still owed.
13. As a baker, I can save a link to the customer's reference image, so that I can find the design they sent me.
14. As a baker, I can record whether an order is pickup or delivery with an address, so that I plan the end of the day correctly.

Not overcommitting — the core of value proposition

15. As a baker, I can see how much work a date already carries before I confirm a new order, so that I don't accept more than I can deliver.
16. As a baker, I can set a maximum workload per day, so that the app has a threshold to measure against.
17. As a baker, I can be warned when an order would push a day past my limit, so that I catch overcommitment at the moment I'm about to say yes.
18. As a baker, I can block out dates I'm unavailable, so that I never accept an order for a day I won't be baking.
19. As a baker, I can set a minimum lead time, so that rush orders get flagged before I agree to them.
20. As a baker, I can ask for the next date with room for a given cake size, so that I can offer the customer an alternative instead of just turning them away.
21. As a baker, I can see which upcoming weeks are already heavily booked, so that I can prepare for peak periods like Christmas.

Working the queue

22. As a baker, I can see everything due today, so that I know what to start on.
23. As a baker, I can see the coming week's orders in due-date order, so that I can plan ahead.
24. As a baker, I can update an order's status (enquiry / confirmed / in progress / ready / collected), so that I can tell which orders are actually locked in.
25. As a baker, I can mark an order collected, so that it leaves my active queue.
26. As a baker, I can see which confirmed orders still have no deposit, so that I chase payment before spending on ingredients.

27. As a baker, I can attach dated prep steps to an order (bake Friday, decorate Saturday), so that I can spread one cake's work across days.

When things change

28. As a baker, I can edit any field of an existing order, so that I can handle a customer changing their mind.
29. As a baker, I can move an order to a different date, so that I can accommodate a postponement.
30. As a baker, I can cancel an order with a reason, so that the day frees up and I still have a record of what happened.
31. As a baker, I can search orders by customer name, so that I can find the right order when someone messages me about it.
32. As a baker, I can undo my last command, so that one mistyped line doesn't destroy an order's details.

Fluent and long-term use

33. As an expert user, I can add a complete order in one line, so that I can enter it while still chatting to the customer.
34. As a baker, I can view a customer's past orders, so that I can reuse what worked and avoid repeating a design.
35. As a baker, I can duplicate a past order as a new one, so that repeat orders take seconds to enter.
36. As a long-time user, I can archive orders older than a chosen date, so that old jobs don't clutter my list.
37. As a baker, I can see how many orders and how much revenue I took in a month, so that I can judge whether the side business is worth continuing.
38. As a returning user, I can see all outstanding orders at a glance after time away, so that I can pick things back up.
39. As a cautious user, I can save my orders to a file I control, so that I don't lose everything if the app breaks.

The core 16

Recording an order (the data everything else depends on)

1. As a baker, I can add an order with a customer name, cake description and due date, so that I stop relying on memory and screenshots.
2. As a baker, I can record the collection date and time, so that I know when the cake must actually be finished, not just which day.
3. As a baker, I can record the size and number of tiers, so that I can judge how much work the cake is.
4. As a baker, I can record flavour, filling and design notes, so that I make the cake the customer actually asked for.
5. As a baker, I can record allergies and dietary restrictions, so that I don't put a customer at risk.
6. As a baker, I can record the quoted price and deposit received, so that I know what is still owed.

Not overcommitting

7. As a baker, I can set a maximum workload per day, so that the app has a threshold to measure against.
8. As a baker, I can see how much work a date already carries before I confirm a new order, so that I don't accept more than I can deliver.
9. As a baker, I can be warned when an order would push a day past my limit, so that I catch overcommitment at the moment I'm about to say yes.
10. As a baker, I can block out dates I'm unavailable, so that I never accept an order for a day I won't be baking.
11. As a baker, I can ask for the next date with room for a given cake size, so that I can offer the customer an alternative instead of just turning them away.

Working the queue

12. As a baker, I can see everything due today, so that I know what to start on.
13. As a baker, I can see the coming week's orders in due-date order, so that I can plan ahead.
14. As a baker, I can update an order's status (enquiry / confirmed / in progress / ready / collected), so that I can tell which orders are actually locked in.

When things change

15. As a baker, I can edit any field of an existing order, so that I can handle a customer changing their mind.
16. As a baker, I can search orders by customer name, so that I can find the right order when someone messages me about it.